

```
krishna $ aws configure
AWS Access Key ID [*****624H]: 
AWS Secret Access Key [*****50tv]: 
Default region name [us-east-1]:
```

Figure 2: Configuring AWS credentials

Once AWS CLI is installed, use the following command to store the AWS security credentials:

```
aws configure
```

The command prompts for AWS access key ID and AWS secret access key like in Figure 2. Supply the details accordingly. You may also supply the default region name.

Packer template

We are all set to write the Packer template. The previous versions of Packer supported JSON for templating. However, the current version recommends using HCL instead. HCL can be used to write templates not only for Packer but also used in other tools of HashiCorp like Terraform.

An HCL template file uses `.hcl` as the extension. In the case of Packer, the file name uses `.pkr.hcl` as the extension. Let us name our template file `ami.pkr.hcl`. The following is the initial template.

```
source "amazon-ecs" "glarimy-ubuntu" {
  region = "us-east-1"
  source_ami = "ami-007855ac798b5175e"
  instance_type = "t2.micro"
  ssh_username = "ubuntu"
  ami_name = "glarimy-ubuntu"
}

build {
  sources = [
    "source.amazon-ecs.glarimy-ubuntu"
  ]
}
```

The above template consists of two blocks, namely, `source` and `build`. An HCL block also contains arguments inside the curly braces. The arguments vary depending on the kind of block.

The Packer `source` block describes the base image from which a new AMI is to be built. In our case, the `source` block specifies the type of source, namely, 'amazon-ecs'. It implies that we are going to use an AMI of EBS type, provided by Amazon. Packer detects this information and tries to interact with AWS. The source block also has a label 'glarimy-ubuntu'. This is just to refer to the block locally in other parts of the template. Together, this block can be referred to

uniquely as `source.amazon-ecs.glarimy-ubuntu`.

The `source` block usually expects us to specify the base image and other basic information.

In our case, we specified that `ami-007855ac798b5175e` is to be used as the base AMI and use the `t2.micro` type of EC2 instance to build the new AMI. We also specified that 'ubuntu' is to be used as the user name for connecting to the instance for further provisioning, if any. Though we specified the region, Packer can actually detect this information along with the AWS security credentials from the local store that we already created. These credentials are required for the Packer to connect to AWS. Finally, the `ami_name` indicates the name that we want to give to the new AMI.

So, in summary, Packer connects to AWS using the security credentials, spins a `t2.micro` instance, deploys `ami-007855ac798b5175e` on it, and builds the new AMI named `glarimy-ubuntu`. Though the new AMI is in no way different from the source AMI, it is a good starting point.

The other block in our template is the `build` block. It does not have any type or label associated with it. It has only one argument, namely, `sources`. Though we specified only `source.amazon-ecs.glarimy-ubuntu` as the source, this argument takes a list of any number of sources. Packer builds the AMIs based on these sources.

Building the AMI

Once the `packer` template file is ready, run the following command in the folder where the template file is located. It downloads any plugins that are required.

```
packer init
```

We can also verify if the template file is valid.

```
packer validate ami.pkr.hcl
```

A valid template file can be formatted properly according to HCL style using the following command:

```
packer fmt ami.pkr.hcl
```

Fine! After initialisation and validation, we are all set to build the actual images.

```
packer build ami.pkr.hcl
```

The command given above takes a few minutes to